

Optimización: El Rol de la Tasa de Aprendizaje

Presentado por:
Rubén Manrique

Universidad de los Andes
Departamento de Ingeniería de Sistemas y Computación

Curso: MLT
Semestre: 2026-10

18 de marzo de 2026

- El entrenamiento de modelos de Machine Learning se reduce fundamentalmente a un problema de optimización no convexa.
- Buscamos minimizar una función de pérdida empírica $L(w)$ parametrizada por los pesos w .

Recordatorio: Actualización en Gradiente Descendente Estocástico (SGD)

En cada iteración t , los parámetros se actualizan en la dirección opuesta al gradiente de la función de pérdida:

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

Componentes:

- $\nabla L(w_t)$: La *dirección* del descenso más pronunciado.
- η (**Tasa de Aprendizaje / Learning Rate**): La *magnitud* del paso que damos en esa dirección.

El Dilema Fundamental de la Tasa de Aprendizaje

La elección del hiperparámetro η es quizás la decisión más influyente en la convergencia de la red. Presenta un trade-off severo:

η Demasiado Grande

- El optimizador da pasos desproporcionados.
- Rebota en puntos diferentes de la superficie del loss.
- **Consecuencia:** Falla en converger, la pérdida oscila violentamente o incluso *diverge* hacia el infinito.

η Demasiado Pequeño

- Los pasos son minúsculos.
- **Consecuencia 1:** Convergencia computacionalmente ineficiente (excesivamente lenta).
- **Consecuencia 2:** Alto riesgo de estancamiento en mínimos locales subóptimos o puntos de silla (*saddle points*).

¿Por qué no mantener η constante durante todo el entrenamiento?

Mantener un η estático asume que la topología del espacio de pérdida es uniforme, lo cual es falso en redes neuronales profundas y modelos complejos. Necesitamos adaptabilidad.

El Equilibrio: Exploración vs. Explotación

- **Fase Temprana (Exploración):** Necesitamos un η relativamente **alto** para escapar de mínimos locales iniciales, cruzar puntos de silla rápidamente y ubicar la cuenca de atracción del mínimo global (o un buen mínimo local).
- **Fase Tardía (Explotación):** A medida que nos acercamos al mínimo, el gradiente no siempre se hace cero en SGD debido al ruido del mini-batch. Necesitamos un η **bajo** para evitar rebotar alrededor del mínimo y lograr asentarnos (settle) en el fondo del valle.

Solución: *Decaimiento (Decay)* o planificación de la tasa de aprendizaje, donde η es una función del tiempo/época t : $\eta(t)$.

En implementaciones como `SGDClassifier` y `SGDRegressor` de `scikit-learn`, el hiperparámetro `learning_rate` controla cómo cambia η a lo largo de las iteraciones t (cada actualización de la muestra).

1. Tasa Constante (`learning_rate='constant'`)

La formulación más simple donde la tasa de aprendizaje no cambia durante el entrenamiento:

$$\eta_t = \eta_0$$

Consideraciones y Uso:

- Exige definir el parámetro `eta0` explícitamente.
- Útil en problemas convexos muy simples y bien condicionados.
- Frecuentemente usado cuando se acopla con optimizadores externos que ya manejan la adaptación del gradiente.
- Típicamente requiere una sintonización manual para evitar divergencia.

2. Escalado Inverso (learning_rate='invscaling')

Reduce gradualmente la tasa de aprendizaje utilizando una función polinomial inversa del tiempo de iteración t :

$$\eta_t = \frac{\eta_0}{t^{\text{power_t}}}$$

Análisis Riguroso:

- **Hiperparámetros:** Requiere definir la tasa inicial η_0 (eta0) y el exponente power_t.
- **Efecto de power_t:** Controla la *velocidad asintótica* de convergencia.
 - Un power_t cercano a 0 se comporta casi como una tasa constante.
 - Un power_t cercano a 1 genera un decaimiento agresivo, útil para forzar la convergencia (explotación) en las fases finales.
- Para convergencia estocástica bajo las condiciones de Robbins-Monro, se requiere que la suma infinita de las tasas diverja, pero la suma de sus cuadrados converja (ej. $\sum \eta_t = \infty$ y $\sum \eta_t^2 < \infty$).

Ejemplo Numérico: Decaimiento Polinomial (invscaling)

Asumamos que configuramos nuestro modelo con:

- $\eta_0 = 0,1$
- $\text{power_t} = 0,5$ (Raíz cuadrada inversa)

La ecuación es: $\eta_t = \frac{0,1}{t^{0,5}} = \frac{0,1}{\sqrt{t}}$

Iteración (t)	Cálculo	Tasa de Aprendizaje (η_t)
$t = 1$	$\frac{0,1}{\sqrt{1}}$	$\eta_1 = 0,1$
$t = 2$	$\frac{0,1}{\sqrt{2}}$	$\eta_2 \approx 0,0707$
$t = 10$	$\frac{0,1}{\sqrt{10}}$	$\eta_{10} \approx 0,0316$
$t = 100$	$\frac{0,1}{\sqrt{100}}$	$\eta_{100} = 0,01$

Observación: La tasa cae rápidamente al principio y luego se estabiliza de forma asintótica, permitiendo un ajuste fino (explotación) en las épocas tardías.

A diferencia de `invscaling` u `optimal`, la estrategia `adaptive` no reduce la tasa basándose únicamente en la iteración t . Actúa observando el comportamiento de la función de pérdida (o la métrica de validación).

Mecanismo de Reducción por Mesetas (Step Decay)

- 1 Inicializa la tasa en $\eta = \eta_0$.
- 2 Mantiene η constante mientras el entrenamiento progrese adecuadamente.
- 3 Si tras `n_iter_no_change` épocas consecutivas, la pérdida de entrenamiento no disminuye en al menos `tol` (o el `score` de validación no aumenta), aplica una penalización multiplicativa:

$$\eta_{nueva} = \frac{\eta_{actual}}{5}$$

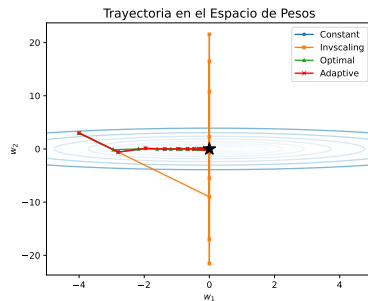
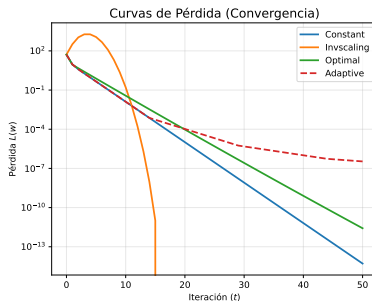
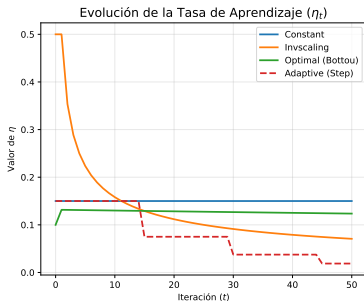
Visualización de Estrategias en 2D

Para ilustrar el comportamiento de cada planificador (schedule), simulamos la optimización sobre una función de pérdida empírica cuadrática y **asimétrica**:

$$L(w_1, w_2) = w_1^2 + 4w_2^2 \implies \nabla L(w) = \begin{bmatrix} 2w_1 \\ 8w_2 \end{bmatrix}$$

Diseño del Experimento:

- **Inicialización:** Partimos desde una zona alejada del mínimo global $(0,0)$, específicamente en $w_0 = [-4,0,3,0]$.

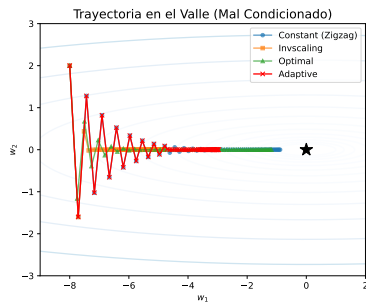
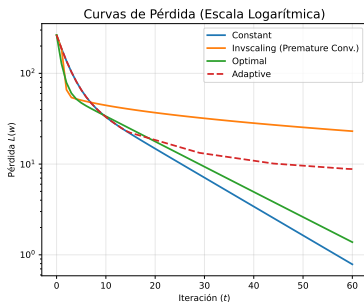
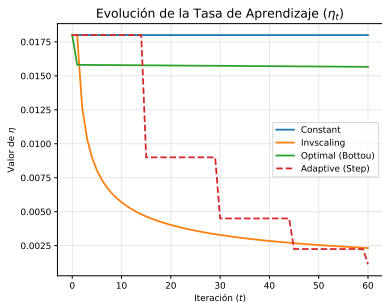


Visualización de Estrategias en 2d

Ahora revisemos una **fuertemente asimétrica**

$$L(w_1, w_2) = w_1^2 + 50w_2^2 \quad \Rightarrow \quad \nabla L(w) = \begin{bmatrix} 2w_1 \\ 100w_2 \end{bmatrix}$$

- **Curvatura:** La pendiente respecto a w_2 es 50 veces más pronunciada que en w_1 .
- **El Efecto Zigzag:** Un η lo suficientemente grande para lograr avanzar en el eje w_1 , causará que la actualización en w_2 cruce violentamente el mínimo y suba por la pared opuesta.
- **Inicialización:** Partimos desde $w_0 = [-8, 0, 2, 0]$, forzando al optimizador a navegar este canal angosto y evidenciando la oscilación.



El Límite de Scikit-Learn: El Problema del η Global

Todas las estrategias vistas (`constant`, `optimal`, `adaptive`) comparten una limitación estructural: aplican el **mismo escalar** η_t a todos los pesos del modelo simultáneamente.

$$w_{t+1} = w_t - \eta_t \nabla L(w_t)$$

¿Por qué es esto un problema?

- **Características Dispersas (Sparse Features):** Si un atributo aparece raramente en los datos (ej. NLP), su peso asociado rara vez recibe gradientes. Deberíamos darle un paso grande cuando aparece. Con un η global que decae con el tiempo, las características raras nunca logran actualizarse significativamente.
- **Valles Mal Condicionados:** Como vimos en la simulación, necesitamos frenar la oscilación en el eje empinado (w_2), pero mantener la velocidad en el eje plano (w_1). Un η global no puede hacer ambas cosas.

La Solución: Adaptabilidad por Parámetro

La clave es tener un vector de tasas de aprendizaje, ajustando un paso distinto para cada dimensión del problema.

1. AdaGrad (Acumulación Histórica)

Escala el gradiente actual inversamente a la raíz cuadrada de la suma de todos los gradientes cuadrados pasados.

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\sum_{i=1}^t g_i^2} + \epsilon} g_t$$

Problema: La suma acumulada crece monótonamente, haciendo que η decaiga a cero prematuramente en redes profundas.

2. RMSProp (Promedio Móvil Exponencial)

Soluciona el problema de AdaGrad usando un decaimiento exponencial (v_t), olvidando gradientes muy antiguos:

$$v_t = \beta v_{t-1} + (1 - \beta) g_t^2 \quad \implies \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t$$

Ejemplo Numérico: AdaGrad vs RMSProp (Iteración 1 y 2)

- ϵ es una constante minúscula (típicamente 10^{-8}) diseñada estrictamente para evitar errores de **división por cero**.
- En la inicialización, o en características muy dispersas (*sparse features* en NLP), el gradiente g_t puede ser exactamente 0. Sin ϵ , el denominador colapsaría, resultando en actualizaciones NaN o Inf que destruyen el modelo.
- Cuando el historial de gradientes v_t es microscópico, ϵ evita que la tasa de aprendizaje efectiva ($\frac{\eta}{\sqrt{v_t + \epsilon}}$) tienda al infinito, limitando el tamaño máximo del paso a $\frac{\eta}{\epsilon}$.

Supongamos un gradiente constante $g = 10$, $\eta = 0,1$, $\epsilon \approx 0$ y para RMSProp $\beta = 0,9$. Inicialmente, $v_0 = 0$.

AdaGrad (Acumulación Total)

Iteración 1:

$$v_1 = v_0 + g_1^2 = 0 + 10^2 = 100$$

$$\text{Paso}_1 = \frac{0,1}{\sqrt{100}} \times 10 = \frac{0,1}{10} \times 10 = 0,1$$

Iteración 2: (Asumiendo $g_2 = 10$)

$$v_2 = v_1 + g_2^2 = 100 + 100 = 200$$

$$\text{Paso}_2 = \frac{0,1}{\sqrt{200}} \times 10 \approx 0,07$$

Problema: v_t crece hacia el infinito. El paso colapsa a 0 rápidamente.

RMSProp (Promedio Móvil)

Iteración 1:

$$v_1 = 0,9(0) + 0,1(10^2) = 10$$

$$\text{Paso}_1 = \frac{0,1}{\sqrt{10}} \times 10 \approx 0,31$$

Iteración 2:

$$v_2 = 0,9(10) + 0,1(10^2) = 9 + 10 = 19$$

$$\text{Paso}_2 = \frac{0,1}{\sqrt{19}} \times 10 \approx 0,23$$

Solución: v_t se estabiliza. El denominador no explota, permitiendo aprendizaje continuo.

El Estándar de la Industria: ADAM (Adaptive Moment Estimation)

ADAM combina magistralmente la heurística de **Momentum** (suavizar la dirección) con **RMSProp** (adaptar el paso por parámetro).

Calcula promedios móviles exponenciales del gradiente (primer momento, m_t) y del gradiente al cuadrado (segundo momento no centrado, v_t):

Primer Momento (Inercia / Momentum)

Acumula la dirección histórica del gradiente.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Típicamente $\beta_1 = 0,9$

Segundo Momento (Escala / RMSProp)

Acumula la magnitud (varianza) del gradiente.

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Típicamente $\beta_2 = 0,999$

La Intuición Física del Primer Momento (m_t)

¿Por qué necesitamos acumular el gradiente histórico en m_t ? La respuesta radica en la física clásica: **Inercia**.

SGD Tradicional

- El paso depende única y exclusivamente del gradiente actual g_t .
- Si $\nabla L(w_t) \approx 0$ (un punto de silla o un mínimo local plano), el optimizador **se detiene**, creyendo que ha convergido.

ADAM / Momentum (La Bola Pesada)

- Actúa como una pesada bola de hierro rodando por la superficie de pérdida.
- Acumula una fracción de los gradientes pasados. Al llegar a una zona plana, su inercia (m_{t-1}) la arrastra hacia adelante, ayudándola a **escapar de mínimos locales**.

En ADAM, el hiperparámetro β_1 (típicamente 0.9) actúa como la "fricción". Determina qué tan rápido el modelo olvida su velocidad anterior.

ADAM: Corrección de Sesgo y Actualización Final

Dado que m_0 y v_0 se inicializan en cero, las estimaciones están fuertemente sesgadas hacia cero en las primeras iteraciones (especialmente v_t porque β_2 es muy cercano a 1).

Paso de Corrección de Sesgo (*Bias Correction*): Se contrarresta matemáticamente dividiendo por el decaimiento acumulado en el tiempo t :

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \text{y} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Regla de Actualización Final de ADAM

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Nota Geométrica: ADAM logra atravesar puntos de silla (*saddle points*) muchísimo más rápido que SGD puro, al acumular momentum en la dirección de curvatura negativa.

Ejemplo Numérico ADAM: La Magia de la Corrección de Sesgo

Veamos qué pasa en la **Iteración 1** con $\eta = 0,1$, $g_1 = 10$, $\beta_1 = 0,9$, $\beta_2 = 0,999$. ($m_0 = 0$, $v_0 = 0$).

1. Cálculo de Momentos (Sesgados hacia cero):

- $m_1 = 0,9(0) + 0,1(10) = 1,0$ (El gradiente real era 10, pero m_1 es solo 1.0)
- $v_1 = 0,999(0) + 0,001(10^2) = 0,1$ (El gradiente al cuadrado era 100, pero v_1 es solo 0.1)

2. Corrección de Sesgo :

- $\hat{m}_1 = \frac{m_1}{1-\beta_1^1} = \frac{1,0}{1-0,9} = \frac{1,0}{0,1} = 10$ (¡Recuperamos la escala real del momento!)
- $\hat{v}_1 = \frac{v_1}{1-\beta_2^1} = \frac{0,1}{1-0,999} = \frac{0,1}{0,001} = 100$ (¡Recuperamos la escala de la varianza!)

3. Actualización Final:

$$w_2 = w_1 - \frac{0,1}{\sqrt{100}} \times 10 = w_1 - \left(\frac{0,1}{10} \times 10 \right) = w_1 - 0,1$$

Visualización con la función cuadrática anterior

Comparativa Visual de Optimizadores

